LAPORAN PRAKTIKUM
STRUKTUR DATA

MODUL 6
DOUBLY LINKED LIST
(BAGIAN PERTAMA)



Disusun Oleh :

NAMA : Fauziah Istiqomah

NIM : 109102530004

Dosen:

Diah Septiana, S.Kom., M.C.S.(AI).

PROGRAM STUDI S-1 SAINS DATA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM
2026

A. Teori Singkat

Doubly linked list adalah linked list yang masing-masing elemennya memiliki dua successor, yaitu successor yang menunjuk ke elemen sebelumnya (*prev*) dan successor yang menunjuk ke elemen sesudahnya (*next*). Doubly linked list juga menggunakan dua buah successor utama pada list, yaitu *First* yang menunjuk ke elemen pertama dan *Last* yang menunjuk ke elemen terakhir. List dinyatakan kosong jika `L.First = Nil`.

Operasi penyisipan (*insert*) pada doubly linked list dilakukan dengan terlebih dahulu mengalokasikan elemen baru, yaitu `P = alokasi(X)`, `P->next = Nil`, dan `P->prev = Nil`. Langkah selanjutnya bergantung pada posisi penyisipan:

Operasi	Langkah

Insert First	`P->next = L.First`
	`(L.First)->prev = P`
	`L.First = P`

Insert Last	`P->prev = L.Last`
	`(L.Last)->next = P`
	`L.Last = P`

Insert After	`P->next = R->next`
	`P->prev = R`
	`(R->next)->prev = P`
	`R->next = P`

Insert Before	`P->prev = Succ->prev`
	`P->next = Succ`
	`(Succ->prev)->next = P`
	`Succ->prev = P`

Operasi penghapusan (*delete*) dilakukan dengan melepas elemen dari list, dengan langkah sebagai berikut:

Operasi	Langkah
Delete First	P = L.First L.First = (L.First)->next P->next = Nil (L.First)->prev = Nil
Delete Last	P = L.Last L.Last = (L.Last)->prev P->prev = Nil (L.Last)->next = Nil
Delete After	R->next = P->next (P->next)->prev = R P->prev = Nil P->next = Nil
Delete Before	P = Succ->prev Succ->prev = P->prev (P->prev)->next = Succ P->next = Nil P->prev = Nil

Proses pencarian, pembaruan, dan penampilan data pada dasarnya sama dengan singly linked list, hanya saja pada doubly linked list penelusuran dapat dilakukan maju maupun mundur.

B. Jawaban Latihan 6.2

Kode (Doublylist.h)

```
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <iostream>
#include <string>
using namespace std;
```

```
#define Nil NULL

struct kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address First;
    address Last;
};

/* ===== Primitif dasar ===== */
void CreateList(List &L);
bool isEmpty(List L);
address alokasi(infotype x);
void dealokasi(address &P);

/* ===== Insert ===== */
void insertFirst(List &L, address P);
void insertLast(List &L, address P);
void insertAfter(List &L, address Prec, address P);
void insertBefore(List &L, address Succ, address P);

/* ===== Delete ===== */
void deleteFirst(List &L, address &P);
```

```
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);
void deleteBefore(address Succ, address &P);

/* ===== Search & View ===== */
address findElm(List L, infotype x);
void printInfo(List L);

#endif
```

Deskripsi

File Doublylist.h berfungsi sebagai deklarasi kerangka struktur data untuk membangun Doubly Linked List dalam bahasa C++. File ini mendefinisikan tipe data bentukan (struct) kendaraan yang memuat komponen nomor polisi, warna, dan tahun pembuatan sebagai isi informasi. Selanjutnya, file ini mendefinisikan elemen list (ElmList) yang memuat penunjuk arah ganda berupa komponen next dan prev, serta mendefinisikan tipe data List yang memuat pointer First dan Last. Selain itu, file ini berisi seluruh prototipe fungsi dan prosedur operasional pendukung, meliputi fungsi dasar (pembuatan list, pengecekan status, alokasi, dan dealokasi memori), operasi penyisipan elemen (di awal, akhir, setelah, dan sebelum node tertentu), operasi penghapusan elemen, operasi pencarian (findElm), serta operasi penampilan data (printInfo). Keseluruhan deklarasi ini menjadi standar prototipe agar fungsi dapat dikenali oleh kompilator sebelum diimplementasikan secara rinci pada file Doublylist.cpp dan dieksekusi melalui main.cpp.

Kode (Doublylist.cpp)

```cpp
#include "Doublylist.h"

void CreateList(List &L) {
    L.First = Nil;
    L.Last = Nil;
}

bool isEmpty(List L) {
    return L.First == Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    P->prev = Nil;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

/* ================= INSERT ================= */

void insertFirst(List &L, address P) {
    if (isEmpty(L)) {
        L.First = P;
        L.Last = P;
    } else {
        P->next = L.First;
        L.First->prev = P;
        L.First = P;
    }
}
```

```
void insertLast(List &L, address P) {
    if (isEmpty(L)) {
        L.First = P;
        L.Last = P;
    } else {
        P->prev = L.Last;
        L.Last->next = P;
        L.Last = P;
    }
}

void insertAfter(List &L, address Prec, address P) {
    if (Prec == Nil) return;
    if (Prec == L.Last) {
        insertLast(L, P);
    } else {
        P->next = Prec->next;
        P->prev = Prec;
        Prec->next->prev = P;
        Prec->next = P;
    }
}

void insertBefore(List &L, address Succ, address P) {
    if (Succ == Nil) return;
    if (Succ == L.First) {
        insertFirst(L, P);
    } else {
        P->prev = Succ->prev;
        P->next = Succ;
        Succ->prev->next = P;
        Succ->prev = P;
    }
}

/* ================= DELETE ================= */
```

```
void deleteFirst(List &L, address &P) {
    P = L.First;
    if (P == Nil) return;
    if (L.First == L.Last) {
        L.First = Nil;
        L.Last = Nil;
    } else {
        L.First = L.First->next;
        L.First->prev = Nil;
        P->next = Nil;
    }
}

void deleteLast(List &L, address &P) {
    P = L.Last;
    if (P == Nil) return;
    if (L.First == L.Last) {
        L.First = Nil;
        L.Last = Nil;
    } else {
        L.Last = L.Last->prev;
        L.Last->next = Nil;
        P->prev = Nil;
    }
}

void deleteAfter(address Prec, address &P) {
    P = Prec->next;
    if (P == Nil) return;
    Prec->next = P->next;
    if (P->next != Nil) {
        P->next->prev = Prec;
    }
    P->next = Nil;
    P->prev = Nil;
}
```

```
void deleteBefore(address Succ, address &P) {
    P = Succ->prev;
    if (P == Nil) return;
    Succ->prev = P->prev;
    if (P->prev != Nil) {
        P->prev->next = Succ;
    }
    P->next = Nil;
    P->prev = Nil;
}

/* ============== SEARCH & VIEW ============== */

address findElm(List L, infotype x) {
    address P = L.First;
    while (P != Nil) {
        if (P->info.nopol == x.nopol) {
            return P;
        }
        P = P->next;
    }
    return Nil;
}

void printInfo(List L) {
    address P = L.Last;
    while (P != Nil) {
        cout << "Nomor Polisi : " << P->info.nopol << endl;
        cout << "Warna        : " << P->info.warna << endl;
        cout << "Tahun        : " << P->info.thnBuat << endl;
        P = P->prev;
    }
}
```

Deskripsi

File Doublylist.cpp berfungsi sebagai pusat implementasi atau rincian algoritma dari seluruh prototipe yang telah dideklarasikan pada file Doublylist.h untuk membangun struktur data Doubly Linked List. Di dalam file ini, dijabarkan secara detail bagaimana komputasi pointer arah ganda (next dan prev) bekerja pada setiap fungsi operasional. Code ini mencakup implementasi operasi dasar seperti inisialisasi list menjadi kosong (CreateList), validasi status list (isEmpty), serta manajemen memori dinamis untuk pembuatan node baru dan pelepasan node yang tidak terpakai (alokasi dan dealokasi). Lebih lanjut, file ini memuat logika penyisipan data (insertFirst, insertLast, insertAfter, insertBefore) dan penghapusan data (deleteFirst, deleteLast, deleteAfter, deleteBefore) yang ditulis lengkap dengan penanganan kondisi khusus, seperti saat list masih kosong, penyisipan di ujung senarai, atau perlindungan agar rantai pointer tidak terputus. Terakhir, terdapat implementasi fitur pencarian (findElm) yang melakukan penelusuran sekuensial untuk mencocokkan nomor polisi kendaraan, serta fitur penampilan data (printInfo) yang secara spesifik menelusuri list dari elemen terakhir (L.Last) mundur ke awal menggunakan pointer prev untuk mencetak seluruh informasi kendaraan ke layar. Secara keseluruhan, fungsinya adalah sebagai mesin penggerak utama (source file) yang berisi logika pemrograman agar struktur data dapat beroperasi dan dieksekusi melalui main.cpp.

Kode (main.cpp)

```
#include "Doublylist.h"

const int JUMLAH_INPUT = 4;

int main() {
    List L;
    infotype x;
```

```
    address P;

    CreateList(L);

    /* ===== Nomor 1 : input data + insertLast ===== */
    for (int i = 0; i < JUMLAH_INPUT; i++) {
        cout << "masukkan nomor polisi: ";
        cin >> x.nopol;
        cout << "masukkan warna kendaraan: ";
        cin >> x.warna;
        cout << "masukkan tahun kendaraan: ";
        cin >> x.thnBuat;

        if (findElm(L, x) != Nil) {
            cout << "nomor polisi sudah terdaftar" << endl;
        } else {
            P = alokasi(x);
            insertLast(L, P);
        }
        cout << endl;
    }

    cout << "DATA LIST 1" << endl << endl;
    printInfo(L);
    cout << endl;

    /* ===== Nomor 2 : findElm ===== */
    cout << "Masukkan Nomor Polisi yang dicari  : ";
    cin >> x.nopol;
    cout << endl;
    P = findElm(L, x);
    if (P != Nil) {
        cout << "Nomor Polisi : " << P->info.nopol << endl;
        cout << "Warna        : " << P->info.warna << endl;
        cout << "Tahun        : " << P->info.thnBuat << endl;
    } else {
        cout << "Data dengan nomor polisi " << x.nopol
```

```
            << " tidak ditemukan." << endl;
    }
    cout << endl;

    /* ===== Nomor 3 : hapus berdasarkan nopol ===== */
    cout << "Masukkan Nomor Polisi yang akan dihapus  : ";
    cin >> x.nopol;
    P = findElm(L, x);
    if (P == Nil) {
        cout << "Data dengan nomor polisi " << x.nopol
             << " tidak ditemukan." << endl;
    } else {
        address D;
        if (P == L.First) {
            deleteFirst(L, D);
        } else if (P == L.Last) {
            deleteLast(L, D);
        } else {
            deleteAfter(P->prev, D);
        }
        dealokasi(D);
        cout << "Data dengan nomor polisi " << x.nopol
             << " berhasil dihapus." << endl;
    }

    cout << endl << "DATA LIST 1" << endl << endl;
    printInfo(L);

    /* bersihkan memori */
    while (!isEmpty(L)) {
        deleteFirst(L, P);
        dealokasi(P);
    }
    return 0;
}
```

Deskripsi

File ini adalah program utama (driver) untuk menguji ADT Doubly Linked List yang sudah dibuat di Doublylist.h dan Doublylist.cpp. Program ini menjawab tiga soal, yaitu memasukkan data kendaraan, mencari data berdasarkan nomor polisi, dan menghapus data berdasarkan nomor polisi.

Nomor 1: Input data kendaraan

Contoh *Output* :

```
masukkan nomor polisi: D001
masukkan warna kendaraan: hitam
masukkan tahun kendaraan: 90

masukkan nomor polisi: D003
masukkan warna kendaraan: putih
masukkan tahun kendaraan: 70

masukkan nomor polisi: D001
masukkan warna kendaraan: merah
masukkan tahun kendaraan: 80
nomor polisi sudah terdaftar

masukkan nomor polisi: D004
masukkan warna kendaraan: kuning
masukkan tahun kendaraan: 90

DATA LIST 1

no polisi : D004
warna     : kuning
tahun     : 90
no polisi : D003
warna     : putih
tahun     : 70
no polisi : D001
warna     : hitam
tahun     : 90
```

Gambar 6-23 *Output* kasus kendaraan

Output

```
PS C:\Users\istiq\OneDrive\STRUKTUR DATA\LAPRAK MODUL 6> .\program
masukkan nomor polisi: D001
masukkan warna kendaraan: hitam
masukkan tahun kendaraan: 90

masukkan nomor polisi: D003
masukkan warna kendaraan: putih
masukkan tahun kendaraan: 70

masukkan nomor polisi: D001
masukkan warna kendaraan: merah
masukkan tahun kendaraan: 80
nomor polisi sudah terdaftar

masukkan nomor polisi: D004
masukkan warna kendaraan: kuning
masukkan tahun kendaraan: 90

DATA LIST 1

Nomor Polisi : D004
Warna        : kuning
Tahun        : 90
Nomor Polisi : D003
Warna        : putih
Tahun        : 70
Nomor Polisi : D001
Warna        : hitam
Tahun        : 90
```

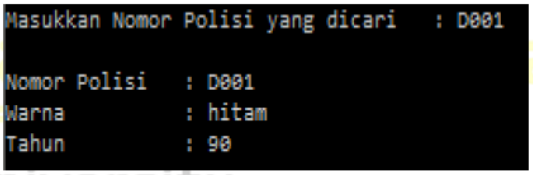
Deskripsi

Program diawali dengan mendeklarasikan variabel L bertipe List, x bertipe infotype untuk menampung data input, dan P bertipe address. List dibuat kosong dengan CreateList. Setelah itu program melakukan perulangan sebanyak empat kali (sesuai konstanta JUMLAH_INPUT) untuk meminta pengguna memasukkan nomor polisi, warna, dan tahun kendaraan. Setiap data yang dimasukkan dicek dengan findElm untuk memastikan nomor polisinya belum terdaftar. Jika sudah ada, program menampilkan pesan “nomor polisi sudah terdaftar” dan data tidak dimasukkan ke list. Jika belum ada, data dialokasikan dengan alokasi lalu disisipkan di akhir list dengan insertLast.

Setelah semua input selesai, isi list ditampilkan dengan printInfo.

Nomor 2: Mencari data berdasarkan nomor polisi

Carilah elemen dengan nomor polisi D001 dengan membuat fungsi baru.
fungsi findElm(L : *List*, x : infotype) : *address*



Gambar 6-24 *Output* mencari nomor polisi

Output

```
Masukkan Nomor Polisi yang dicari  : D001

Nomor Polisi : D001
Warna        : hitam
Tahun        : 90
```

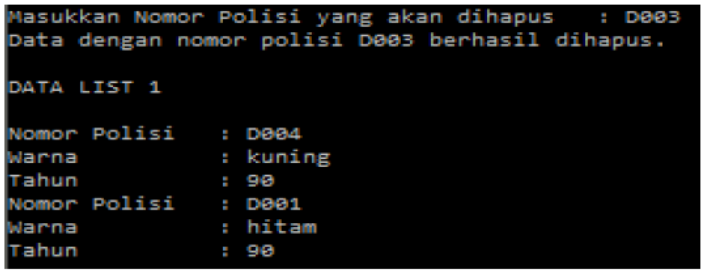
Deskripsi

Program meminta pengguna memasukkan nomor polisi yang dicari, lalu memanggil fungsi findElm. Jika elemen ditemukan, hasilnya berupa alamat elemen sehingga nomor polisi, warna, dan tahun kendaraan tersebut dapat ditampilkan melalui P->info. Jika tidak ditemukan, findElm mengembalikan Nil dan program menampilkan pesan bahwa data dengan nomor polisi tersebut tidak ditemukan.

Nomor 3: Menghapus data berdasarkan nomor polisi

Hapus elemen dengan nomor polisi D003 dengan *procedure delete*.

- *procedure* deleteFirst(input/output L : *List*, P : *address*)
- *procedure* deleteLast(input/output L : *List*, P : *address*)
- *procedure* deleteAfter(input Prec : *address*, input/output P : *address*)



Gambar 6-25 *Output* menghapus data nomor polisi

Output

```
Masukkan Nomor Polisi yang akan dihapus  : D003
Data dengan nomor polisi D003 berhasil dihapus.

DATA LIST 1

Nomor Polisi : D004
Warna        : kuning
Tahun        : 90
Nomor Polisi : D001
Warna        : hitam
Tahun        : 90
```

Deskripsi

Program meminta pengguna memasukkan nomor polisi yang akan dihapus, lalu mencari elemennya dengan findElm. Jika tidak ditemukan, program menampilkan pesan bahwa data tidak ditemukan. Jika ditemukan, prosedur

penghapusan dipilih sesuai posisi elemen. Jika elemen berada di awal list, digunakan deleteFirst. Jika berada di akhir list, digunakan deleteLast. Jika berada di tengah, digunakan deleteAfter dengan elemen sebelumnya (P->prev) sebagai acuan. Elemen yang sudah dilepas dari list kemudian dihapus dari memori dengan dealokasi, lalu program menampilkan pesan bahwa data berhasil dihapus dan menampilkan kembali isi list dengan printInfo.

Di akhir program, seluruh elemen yang masih tersisa di list dihapus satu per satu dengan deleteFirst dan dealokasi supaya tidak ada memori yang tertinggal (*memory leak*), kemudian program selesai dengan return 0.